

OPERATING SYSTEMS PROJECT PROPOSAL

1. Project Title

Design and Implementation of a University Exam Result Portal Using Reader–Writer Synchronization with a GUI and Multi-Process Shared Memory (SysV IPC)

2. Group Members & Course Information

Name: Muhammad Ali

Roll Number: 23K-0015

Course: Operating Systems

Semester: Spring 2026

3. Introduction / Background

Modern university systems face intense concurrent load during result announcements, where many students access records while administrators correct or update data. The classical **Reader–Writer Problem** models this challenge, where multiple readers can access shared data simultaneously, but writers require exclusive access to ensure data consistency.

Operating systems provide thread-level and inter-process synchronization mechanisms to maintain correctness and prevent conflicts. This project extends a result portal simulation beyond a single-process threaded model by introducing a graphical user interface (GUI) with separate student and administrator workflows, while implementing multi-process coordination using explicit system-level APIs such as process creation and shared memory.

4. Problem Statement

In a university result portal, concurrent reads must not observe inconsistent state while a write operation is in progress. Similarly, write operations must not corrupt records or create partial updates. Without proper synchronization, race conditions, inconsistent reads, and system instability may occur.

The challenge is to design a system that:

- Allows controlled concurrent reading for students.
- Grants writers (administrators) exclusive access during updates.
- Avoids deadlock and minimizes starvation through a proper scheduling policy.
- Provides separate GUI views for students and administrators.
- Ensures concurrency correctness using low-level operating system mechanisms such as processes and inter-process communication (IPC).

5. Objectives of the Project

The main objectives of this project are:

- To implement the **Reader–Writer Problem** for a university result portal using **C/C++ on Linux**.
- To develop a **GUI-based application** with separate interfaces for students and administrators.
- To coordinate concurrency using **inter-process shared memory** through `shmget()`, `shmat()`, and related system calls.
- To use explicit UNIX/POSIX system programming functions, including:
 - `fork()` for process creation
 - `read()` and `write()` for IPC and file operations
 - `pipe()`, `open()`, and `close()`
- To maintain data consistency in file storage using safe update strategies.
- To analyze system behavior under read-heavy and write-heavy workloads.

6. Scope of the Project

This project simulates a university result portal with a focus on operating system concepts such as concurrency, synchronization, and IPC.

In Scope

- GUI application with separate student and admin views.
- Backend concurrency using multiple processes.
- Shared memory segments created and managed via `shmget()`.
- Demonstrable use of:
 - `fork()`
 - `pipe()`
 - `read()`
 - `write()`
- File-based storage for credentials and exam results.
- Safe and consistent update mechanisms.

Out of Scope

- Production-grade security features such as encryption and external authentication servers.
- Distributed or network-based deployment.
- Cloud integration or web-based hosting.

7. Literature Review / Related Work

The **Reader–Writer Problem** is a classical synchronization problem in operating systems. Common scheduling approaches include:

- **Reader Preference:** Prioritizes readers but may starve writers.
- **Writer Preference:** Prioritizes writers to avoid writer starvation.
- **Fair Scheduling:** Attempts to balance both readers and writers.

Modern software systems often combine concurrency control with process isolation. GUI applications typically separate the user interface from worker processes, while operating systems coursework commonly uses **System V IPC (shared memory and semaphores)** as a standard method for process communication and synchronization.

This project follows that model by storing synchronization metadata and shared records in shared memory while explicitly enforcing reader–writer rules.

8. Proposed System / Methodology

User Roles

Student (Reader)

- Authenticate using stored credentials.
- View personal exam results.
- Perform read-only operations concurrently with other students.

Administrator (Writer)

- Authenticate using admin credentials.
- Add or update student records.
- Receive exclusive access during modifications.

Execution Model

- A **GUI front-end process** presents student and administrator interfaces.
- The GUI communicates with backend worker processes created using `fork()`.
- `pipe()` channels transfer requests and responses using `read()` and `write()`.
- A **System V shared memory segment** (`shmget()`, `shmat()`, `shmdt()`, `shmctl()`) stores:
 - Active reader count
 - Writer lock flags
 - Shared result data (if required)
- File persistence uses:
 - `open()`
 - `read()`
 - `write()`
 - `close()`

Synchronization Strategy

Since regular mutexes are insufficient across separate processes, this project will use:

- **System V Semaphores** (`semget()`, `semop()`)

- Shared memory synchronization variables
- A **Writer Preference Policy** to reduce writer starvation

Optional Features

- Operation logging with timestamps
- Wait-time and contention analysis
- Performance metrics dashboard

9. System Architecture / Design

Main Modules

1. GUI Module

- Student dashboard
- Administrator dashboard
- Input validation
- Result display and status messages

2. IPC Module

- Pipe-based communication
- Message serialization
- Request/response handling using `read()` and `write()`

3. Process Management Module

- Worker process creation using `fork()`
- Parent-child coordination

4. Shared Memory Module

- Memory allocation via `shmget()`
- Memory attachment/detachment via `shmat()` and `shmdt()`
- Cleanup using `shmctl()`

5. Reader–Writer Synchronization Module

- Concurrent reader access
- Exclusive writer access
- Starvation prevention policy

6. Storage Module

- Raw file I/O for credentials and result records
- Safe file update strategies

System Workflow

1. User selects role (**Student/Admin**) in the GUI.
2. GUI sends request to backend through a pipe.
3. Backend checks synchronization permissions.
4. Readers access data concurrently if allowed.
5. Writers receive exclusive access.
6. Backend returns results to GUI.
7. GUI displays output.

10. Operating System Concepts Used

The project demonstrates the following operating system concepts:

- **Process Creation:** `fork()`
- **Inter-Process Communication (IPC):**
 - `pipe()`
 - `read()`
 - `write()`
- **Shared Memory:**
 - `shmget()`
 - `shmat()`
 - `shmdt()`
 - `shmctl()`
- **Synchronization:**
 - `semget()`

- semop()
 - Reader–Writer logic
- **Critical Section Management**
- **Deadlock and Starvation Prevention**
- **Low-Level File I/O:**
 - open()
 - read()
 - write()
 - close()

11. Tools and Technologies

Programming Language: C or C++

GUI Toolkit: GTK or Qt (Linux)

Operating System APIs:

- fork()
- pipe()
- read()
- write()
- open()
- close()
- shmget()
- shmat()
- shmdt()
- shmctl()
- semget()
- semop()

Build Tools: GCC / G++ and Makefile

Development Environment: Ubuntu Linux, VS Code or equivalent

12. Implementation Plan & Timeline

- Finalize system architecture
- Define IPC message formats
- Design shared memory layout
- Choose GUI toolkit
- Create GUI skeleton
- Implement backend worker processes using `fork()`
- Establish pipe-based communication
- Integrate request handling
- Implement shared memory using `shmget()`
- Develop reader–writer synchronization logic
- Test concurrent behavior
- Implement file persistence
- Add logging and performance metrics
- Perform stress testing
- Finalize GUI and documentation

13. Expected Results and Conclusion

The completed system is expected to provide:

- A functional GUI with separate student and admin workflows.
- Correct implementation of the **Reader–Writer synchronization problem**.
- Multi-process coordination using **shared memory and IPC**.
- Reliable file storage without race conditions or corruption.
- Performance measurements under different workloads.

Conclusion

This project strengthens understanding of advanced operating system concepts by combining theoretical synchronization with practical implementation. It demonstrates the use of **process creation, shared memory, semaphores, IPC, and low-level file I/O** in a realistic university result portal application, bridging the gap between academic theory and practical systems software engineering.